Assignment 3

1701ICT— Creative Coding
School of ICT
Griffith University
Trimester 1, 2025

Assignment 3 (40%) is due at the end of Week 11 with demonstrations to be held in your week 12 workshop. **(11:59pm Friday 23rd May)**

Instructions

- **Due**: 11:59pm Friday 23rd May)
- **Marks**: 40% of your overall grade
- **Late Submissions**: Late submission is allowed but a penalty applies. The penalty is defined as the reduction of the mark allocated to the assessment item by 5% of the total weighted mark for the assessment item, for each day that the item is late. Assessment items submitted more than five days after the due date will be awarded zero marks.
- **Extensions**: You can request for an extension of time on one of two grounds;
    - medical
    - other (e.g., family or personal circumstances, employment-related circumstances, unavoidable commitments).

  All requests must be made through the myGriffith portal.

- **Individual Work**: You must complete this assignment individually.
- **Presentation**: You must present/demonstrate your work in your week 12 workshop to a teaching team member. Your work will not be marked if you do not present it.
- **Other**: This assignment is a mandatory pass component. You must attain 40% (16/40) to be eligible to pass the course.

Overview

The purpose of this assignment is to assess your ability to implement complex interactions between separate modules using JavaScript and p5.js. Each Question should be submitted on L@G in a separate .js file (titled q1.js, q2.js etc.), but you do not need to submit html and CSS files. Some of the questions may require you to investigate some concepts outside of the lecture notes. Please make sure that all code that you submit is your own and is not taken/copied from anywhere else.

Game **(due end of Week 11, 11:59pm Friday 23rd May)**

You will be creating your own version of the game 'R-Type'. This is provided as inspiration, but the possibilities/theming are vast.

R-type II (1989)

R-type III Evolved (2023 PS5)

The player ship should be controlled by keyboard, and at a minimum you should have to fight waves of enemies that come from the side (and maybe top/bottom) of the screen. https://www.youtube.com/watch?v=pluqUkjZycQ provides some more inspiration.

Requirements

1. Multiple Scenes

The game must involve at least 4 scenes, there must be a:

- Loading / splash scene
- Main menu scene
- Main game scene
- Leaderboard scene

You should be able to navigate between the scenes

Note: each scene needs to only be navigable by at least one other scene (with the exception of the loading scene, which can be a time-oriented screen)

2. Loops & Arays

The game should use loops where appropriate to iterate over arrays/groups. The game must incorporate at least 3 arrays, with at least 1 of the arrays being used to manage a list of objects on the screen (such as incoming missiles). p5.play Groups will count as arrays

3. Interactivity

There must exist player interaction using a keyboard. The game needs to handle at least 4 keyboard keys. Using mouse input is optional.

4. Images

There must be at least 1 image present in the final product (separate to sprites) that is drawn at an appropriate size (suggestion: main menu background, game background, leaderboard imagery or other cases where sprite functionality is not required).

Note: it is up to you which scene you incorporate the image in.

5. Sprites

The p5.play library (version 2) must be incorporated with a minimum of 3 sprites used. The sprites must have collision detection (between sprites) and must move independently. Sprites must be animated (i.e. a moving object, changing size or colour)

6. Video & Sound

The game should use at least 3 sounds and 1 video. Hint: you can use sounds for shooting and video to introduce the game in the main screen.

7. GUI Input

At least one type of GUI input is required (e.g. text input, slider, button, text box). (Hint: Text input could be useful for high score entry, or a button to start the game)

8. Data

The program needs to read in data from a JSON file where each object must have at least 3 properties. This data must be presented in one of the scenes.

Hint: if your themes / ideas of the main game scene conflict/don't work well with using external data, use the data in the leaderboard scene!

9. Stability

The game must feel stable to play. Your game should not feel 'buggy'.

10. Creativity

You must produce creative work that adds complexity to your game.

Examples of creative work that add complexity to your game include:

- Having multiple types of enemies
    - You may have other enemy types that enter to make the game harder or boss fights.
- Having a dynamic background
    - Having a scene with a day/night cycle? Or Stars (sprites or images) may traverse down the screen in a straight line, being generated at random rates & positions from the top of the screen
- Having multiple weapon types / modes of fire

    - Your ship may fire in a straight line in its unaltered state, but when picking up a power up (that appears randomly or based on some event kill) the ship has an altered mode of fire, such as shooting in multiple directions.
    - You could add super weapons, or other beneficial additions to the defenders (shields? Power-ups?)
- UI & Fonts
    - You may download and use custom fonts that better fit your theme.
- Score & Sound Effects
    - A special sound every 10 enemies destroyed and add relevant bonus scoring.
- Enemy health
    - Enemies only fall after being hit 3 times. Use an array to store the health of individual enemies and update it upon collision detection.
- Ability to change the difficulty of game
    - Could be as simple as having two difficulty options that change the number of lives available, or as creative as having increased speed of incoming fire / increased toughness of opponents.

You are encouraged to create your own assets (i.e. opponents or laser fire) on top of using public assets from the internet.

Note: the ability to creatively expand your game is highly dependent on having basic functionality, such as interactivity.

11. Code

You must produce clean, well-written code. This encompasses indentation, meaningful naming of functions & variables, use & overuse of absolute values.